

Sistema de Gestión de Mantenimiento de Equipos

SGME

Documentación Técnica

Tecnologías: Python 3.8+ · SQLite · Flask

Arquitectura: Modelo-Vista-Controlador (MVC)

Base de datos: SQLite 3

Presentado por:

Emanuel Cardenas

Samuel Manchola

Sebastian Medina

Mayo 2026

Índice

1. Introducción	2
1.1. Objetivos del Sistema	2
1.2. Alcance	2
2. Diagramas	3
2.1. Diagrama Entidad-Relación.....	3
2.2. Descripción de las Relaciones	3
2.3. Diagrama Casos de Uso	4
2.4. Diagrama de Clases	5
2.5. Diagrama de Despliegue	5
2.6. Restricciones de Integridad.....	5
2.7. Índices.....	6
3. Arquitectura del Sistema	7
3.1. Diagrama de Capas.....	7
3.2. Descripción de las Capas.....	7
3.2.1. Capa de Presentación (Vista)	7
3.2.2. Capa de Lógica de Negocio (Modelo)	8
3.2.3. Capa de Datos	8
3.3. Reglas de Negocio	8
4. Configuración Requerida	9
4.1. Requisitos del Sistema	9
4.2. Dependencias.....	9
4.3. Instrucciones de Instalación	9
4.4. Instrucciones de Ejecución	10
4.4.1. Opción 1: Interfaz Web (recomendada)	10
4.4.2. Opción 2: Interfaz de Consola	10
5. Descripción de Módulos	11
5.1. Estructura de Archivos.....	11
5.2. Módulo: schema.sql.....	11
5.3. Módulo: database.py	12
5.4. Módulo: models/equipo.py	12
5.5. Módulo: models/tecnico.py	12
5.6. Módulo: models/orden_trabajo.py	13
5.7. Módulo: reports/historial.py	13
5.8. Módulo: reports/reportes.py.....	13
5.9. Módulo: app.py (Controlador Web)	14
5.10. Módulo: views/menu.py (Controlador Consola)	14
5.11. Módulo: views/tabla.py	14
6. Datos de Ejemplo	15

1. Introducción

El **Sistema de Gestión de Mantenimiento de Equipos (SGME)** es una aplicación de software funcional desarrollada en **Python** con **SQLite** que gestiona el ciclo de vida completo del mantenimiento de equipos industriales, incluyendo planificación, ejecución, historial y generación de reportes.

1.1. Objetivos del Sistema

- Gestionar el inventario de equipos industriales con su ciclo de vida.
- Administrar el registro de técnicos especializados.
- Controlar las órdenes de trabajo desde su creación hasta su resolución.
- Mantener un historial completo de mantenimientos realizados.
- Generar reportes especiales para la toma de decisiones.
- Aplicar reglas de negocio que garanticen la integridad de los datos.

1.2. Alcance

El sistema ofrece dos interfaces de usuario:

1. **Interfaz de consola** (main.py): Menús interactivos basados en texto.
2. **Interfaz web** (app.py): Aplicación web moderna con Flask, accesible desde el navegador.

Ambas interfaces comparten la misma base de datos y lógica de negocio.

2. Diagramas

2.1. Diagrama Entidad-Relación

El sistema consta de tres entidades principales relacionadas entre sí:

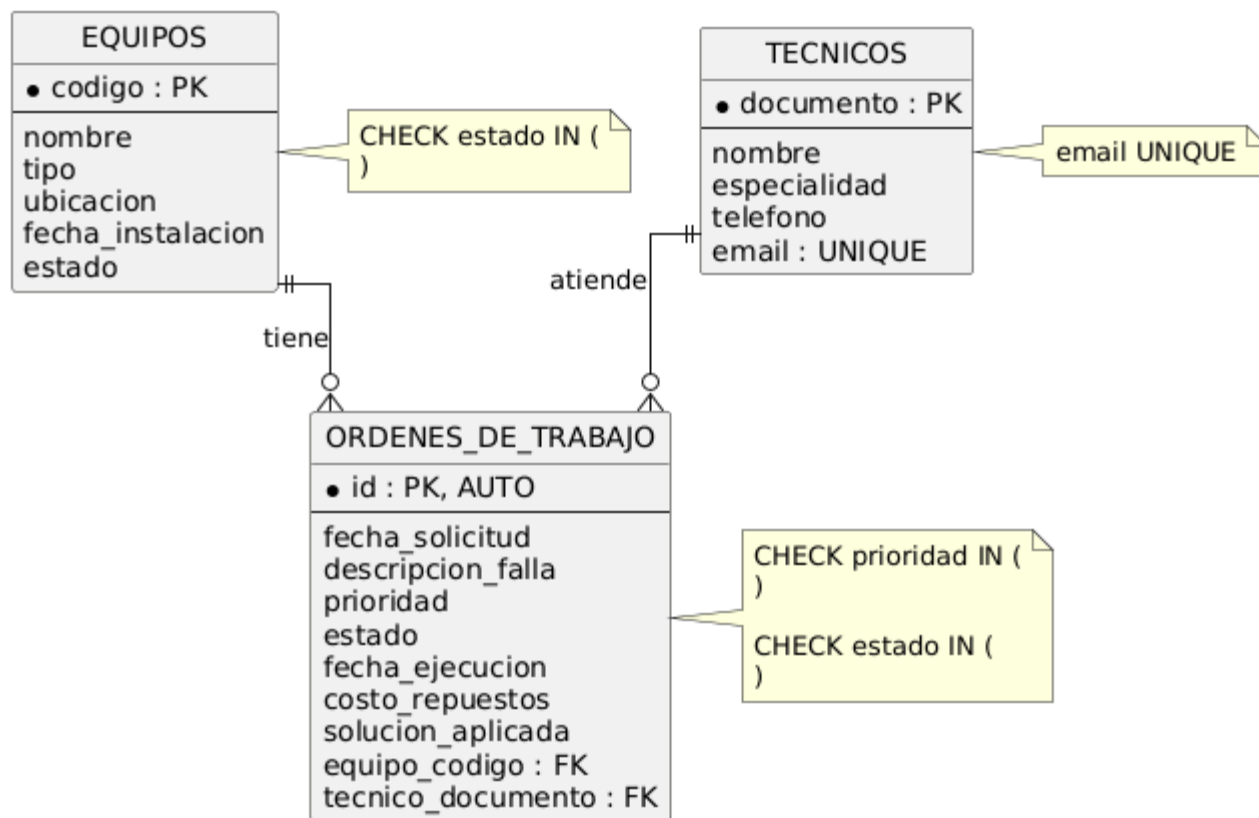


Figura 1: Diagrama Entidad-Relación del Sistema SGME

2.2. Descripción de las Relaciones

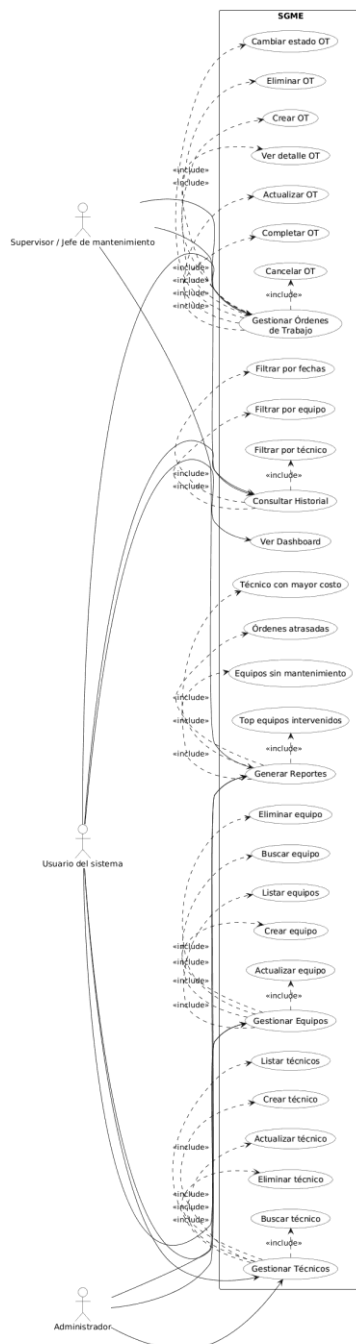
Relación	Cardinalidad	FK	Restricción
Equipos → Órdenes	1:N	equipo_codigo	ON DELETE RESTRICT
Técnicos → Órdenes	1:N	tecnico_documento	ON DELETE RESTRICT

Cuadro 1: Relaciones entre entidades

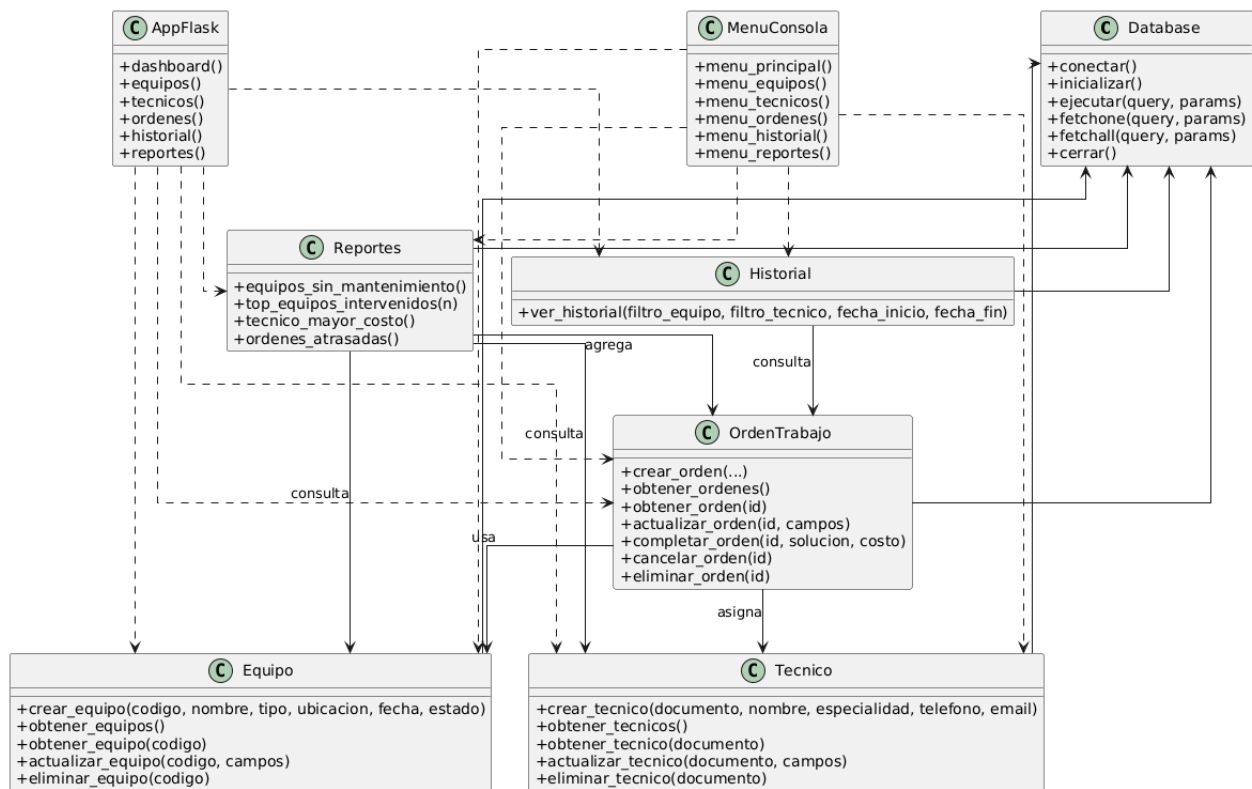
- Un **equipo** puede tener muchas órdenes de trabajo (1:N).

- Un **técnico** puede ser asignado a muchas órdenes de trabajo (1:N).
- Una **orden de trabajo** pertenece a exactamente un equipo y un técnico.
- Las claves foráneas usan ON DELETE RESTRICT para impedir la eliminación de equipos o técnicos con órdenes asociadas.

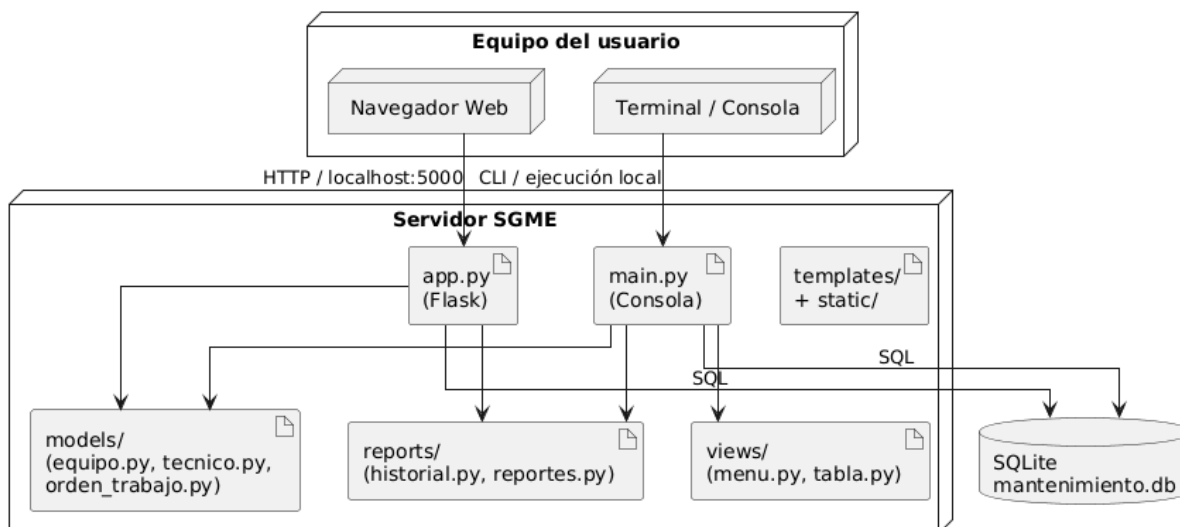
2.3 Diagrama Casos de Uso



2.4 Diagrama de Clases



2.5 Diagrama de Despliegue



2.6 Restricciones de Integridad

Tabla	Tipo	Restricción
equipos	CHECK	estado IN ('operativo', 'en mantenimiento', 'fuera de servicio')
ordenes_trabajo	CHECK	prioridad IN ('alta', 'media', 'baja')

ordenes_trabajo	CHECK	estado IN ('pendiente', 'en proceso', 'completada', 'cancelada')
tecnicos	UNIQUE	email debe ser único

Cuadro 2: Restricciones de integridad referencial y de dominio

2.7 Índices

Índice	Columna(s)	Propósito
idx_equipos_estado	equipos(estado)	Filtrar equipos por estado
idx_ordenes_equipo	ordenes_trabajo(equipo_codigo)	JOINS con equipos
idx_ordenes_tecnico	ordenes_trabajo(tecnico_documento)	JOINS con técnicos
idx_ordenes_estado	ordenes_trabajo(estado)	Filtrar OTs por estado
idx_ordenes_fecha	ordenes_trabajo(fecha_solicitud)	Consultas por fecha

Cuadro 3: Índices definidos para optimización de consultas

3 Arquitectura del Sistema

El sistema implementa una arquitectura de **tres capas** basada en el patrón **Modelo-Vista-Controlador (MVC)**, separando claramente las responsabilidades de datos, lógica de negocio y presentación.

3.3 Diagrama de Capas

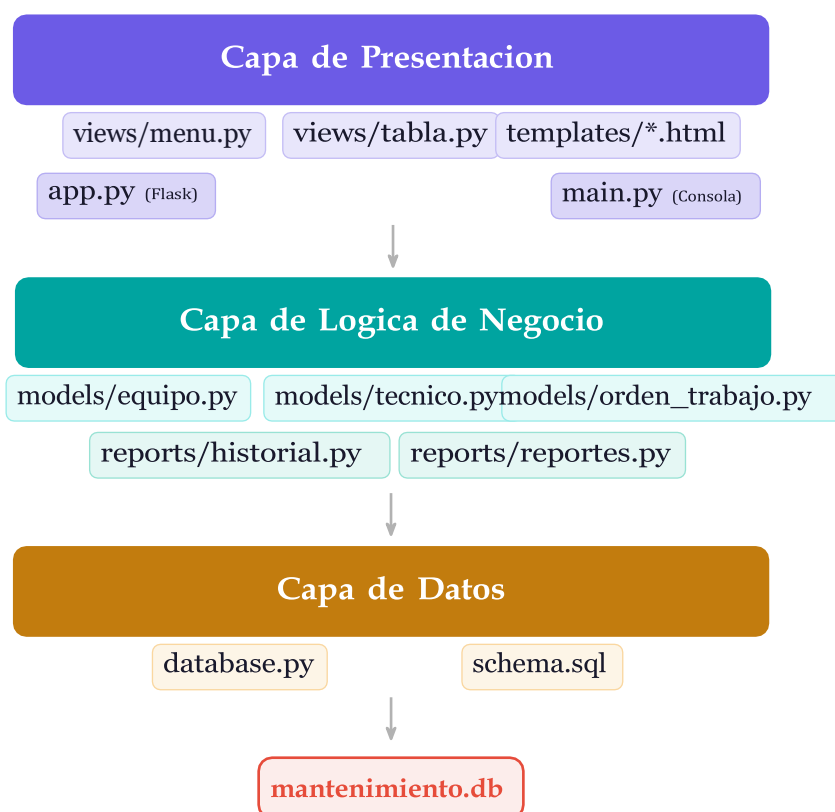


Figura 2: Arquitectura de tres capas del sistema SGME

3.4 Descripción de las Capas

3.4.1 Capa de Presentación (Vista)

Es la capa que interactúa directamente con el usuario. Existen dos implementaciones:

3.4.1.1 Interfaz Web (Flask): Plantillas HTML con CSS moderno, accesible desde `http://localhost:50`. Los archivos se encuentran en `templates/` y `static/`.

3.4.1.2 Interfaz de Consola: Menús interactivos con tablas ASCII formateadas, implementados en `views/menu.py` y `views/tabla.py`.

Esta capa **no contiene lógica de negocio**; solo se encarga de mostrar información y capturar la entrada del usuario.

3.4.2 Capa de Lógica de Negocio (Modelo)

Contiene todas las reglas de negocio y operaciones CRUD:

3.4.2.1 **Modelos** (models/): Operaciones de creación, lectura, actualización y eliminación de equipos, técnicos y órdenes de trabajo. Aquí se validan las reglas de negocio.

3.4.2.2 **Reportes** (reports/): Consultas SQL especializadas para historial y reportes avanzados.

3.4.3 Capa de Datos

Gestiona la persistencia y el acceso a la base de datos:

3.4.3.1 `database.py`: Clase Singleton que gestiona la conexión SQLite, ejecuta consultas y controla transacciones.

3.4.3.2 `schema.sql`: Script DDL con la definición de tablas, índices, restricciones y datos de ejemplo.

3.4.3.3 `mantenimiento.db`: Archivo de base de datos SQLite generado automáticamente.

3.5 Reglas de Negocio

ID	Módulo	Descripción
RN-01	<code>equipo.py</code>	No se puede eliminar un equipo que tenga órdenes de trabajo asociadas. Se verifica mediante COUNT antes del DELETE, reforzado por FK ON DELETE RESTRICT.
RN-02	<code>orden_trabajo.py</code>	No se puede asignar un técnico que no esté registrado en el sistema. Se verifica la existencia del técnico en la tabla antes de crear la orden.
RN-03	<code>orden_trabajo.py</code>	Al completar una orden de trabajo, el equipo asociado cambia automáticamente a estado “operativo”. Se ejecuta dentro de la misma transacción.

Cuadro 4: Reglas de negocio implementadas

4 Configuración Requerida

4.3 Requisitos del Sistema

Componente	Requisito
Sistema Operativo	Windows 10/11, Linux o macOS
Python	Versión 3.8 o superior
SQLite	Incluido con Python (módulo sqlite3)
Flask	Versión 3.x (para la interfaz web)
Navegador	Cualquier navegador moderno (Chrome, Firefox, Edge)

Cuadro 5: Requisitos mínimos del sistema

4.4 Dependencias

Paquete	Tipo	Uso
sqlite3	Estándar (incluido)	Motor de base de datos
os	Estándar (incluido)	Operaciones del sistema de archivos
sys	Estándar (incluido)	Configuración del intérprete
datetime	Estándar (incluido)	Manejo de fechas
flask	Externo (pip)	Framework web para la interfaz gráfica

Cuadro 6: Dependencias del proyecto

Nota

Para la interfaz de consola (main.py), **no se requiere ninguna dependencia externa**. Todo funciona con la librería estándar de Python.

4.5 Instrucciones de Instalación

1. Verificar versión de Python:

```
1 python --version
2 # Debe mostrar Python 3.8 o superior
```

2. Instalar Flask (solo para interfaz web):

```
1 pip install flask
```

3. Ubicarse en el directorio del proyecto:

```
1 cd "proyecto final paradigmas"
```

4.6 Instrucciones de Ejecución

4.6.1 Opción 1: Interfaz Web (recomendada)

```
1 python app.py
```

Al ejecutar, el sistema:

1. Inicializa la base de datos automáticamente (si no existe).
2. Inicia el servidor web en `http://localhost:5000`.
3. Abrir esa URL en el navegador para acceder al sistema.

4.6.2 Opción 2: Interfaz de Consola

```
1 python main.py
```

Muestra un menú interactivo en la terminal con opciones numeradas.

Importante

La primera ejecución crea el archivo `mantenimiento.db` y carga los datos de ejemplo automáticamente. Las ejecuciones posteriores detectan que la base de datos ya existe y no duplican los datos.

5 Descripción de Módulos

5.3 Estructura de Archivos

```

proyecto final paradigmas/
|-- schema.sql                # Script DDL + datos de ejemplo
|-- database.py               # Conexion y gestion de BD (
    Singleton )
|-- main.py                   # Punto de entrada (consola)
|-- app.py                    # Punto de entrada (web Flask)
|-- models/
|   |-- __init__.py
|   |-- equipo.py             # CRUD de equipos
|   |-- tecnico.py            # CRUD de tecnicos
|   |-- orden_trabajo.py      # CRUD avanzado de ordenes
|-- views/
|   |-- __init__.py
|   |-- menu.py               # Menus interactivos de consola
|   |-- tabla.py              # Formateo de tablas ASCII
|-- reports/
|   |-- __init__.py
|   |-- historial.py          # Historial de mantenimiento
|   |-- reportes.py           # Reportes especiales
|-- templates/                # Plantillas HTML (Flask)
|   |-- base.html             # Layout base con sidebar
|   |-- dashboard.html        # Pagina principal
|   |-- equipos/              # CRUD de equipos
|   |-- tecnicos/            # CRUD de tecnicos
|   |-- ordenes/              # CRUD de ordenes
|   |-- historial.html        # Historial con filtros
|   |-- reportes.html         # Reportes especiales
|-- static/
|   |-- style.css              # Estilos CSS (diseno oscuro)
|-- mantenimiento.db          # Base de datos SQLite (
    autogenerada )

```

5.2. Módulo: schema.sql

Responsabilidad	Definición del esquema de base de datos
Tipo	Script SQL
Contenido	Sentencias CREATE TABLE para las 3 tablas del sistema, CREATE INDEX para los 5 índices de optimización, restricciones CHECK y FOREIGN KEY, e instrucciones INSERT con datos de ejemplo (15 equipos, 12 técnicos, 15 órdenes).

5.3. Módulo: `database.py`

Responsabilidad	Gestión de la conexión y operaciones sobre la base de datos
Patrón	Singleton
Clase	Database
Métodos	<code>conectar()</code> : Establece conexión SQLite con <code>PRAGMA foreign_keys = ON</code> y <code>check_same_thread=False</code> . <code>inicializar()</code> : Ejecuta <code>schema.sql</code> si las tablas no existen. <code>ejecutar(query, params)</code> : INSERT/UPDATE/DELETE con commit y rollback. <code>fetchone(query, params)</code> : Retorna un registro. <code>fetchall(query, params)</code> : Retorna todos los registros. <code>cerrar()</code> : Cierra la conexión.

5.4. Módulo: `models/equipo.py`

Responsabilidad	Operaciones CRUD para la entidad Equipos
Regla de negocio	RN-01: No eliminar equipos con OTs asociadas
Funciones	<code>crear_equipo(codigo, nombre, tipo, ubicacion, fecha, estado)</code> : Crea un equipo validando el estado. <code>obtener_equipos()</code> : Retorna todos los equipos ordenados por código. <code>obtener_equipo(codigo)</code> : Busca un equipo por su clave primaria. <code>actualizar_equipo(codigo, **campos)</code> : Actualiza campos específicos. <code>eliminar_equipo(codigo)</code> : Verifica OTs asociadas antes de eliminar.

5.5. Módulo: `models/tecnico.py`

Responsabilidad	Operaciones CRUD para la entidad Técnicos
Validaciones	Email único, protección contra eliminación con OTs activas
Funciones	<code>crear_tecnico(documento, nombre, especialidad, tel, email)</code> : Crea un técnico con validación de unicidad. <code>obtener_tecnicos()</code> : Lista todos los técnicos. <code>obtener_tecnico(documento)</code> : Busca por documento. <code>actualizar_tecnico(documento, **campos)</code> : Actualiza datos. <code>eliminar_tecnico(documento)</code> : Verifica OTs activas antes de eliminar.

5.6. Módulo: `models/orden_trabajo.py`

Responsabilidad Reglas	CRUD avanzado de Órdenes de Trabajo con reglas de negocio RN-02 (técnico registrado) y RN-03 (equipo operativo al completar)
Funciones	<code>crear_orden(...)</code>: Valida equipo y técnico, crea OT, cambia equipo a “en mantenimiento”. <code>obtener_ordenes()</code>: Lista OTs con JOIN a equipos y técnicos. <code>obtener_orden(id)</code>: Detalle completo de una OT. <code>actualizar_orden(id, **campos)</code>: Modifica campos permitidos. <code>completar_orden(id, solucion, costo)</code>: Cierra la OT, registra fecha, cambia equipo a “operativo”. <code>cancelar_orden(id)</code>: Cancela una OT (no completadas). <code>eliminar_orden(id)</code>: Solo permite eliminar OTs pendientes o canceladas.

5.7. Módulo: `reports/historial.py`

Responsabilidad	Consultas de solo lectura sobre el historial de mantenimiento
Funciones	<code>ver_historial(filtro_equipo, filtro_tecnico, fecha_inicio, fecha_fin)</code>: Retorna OTs completadas con JOINS a equipos y técnicos, aplicando filtros dinámicos opcionales. Ordenadas por fecha de ejecución descendente.

5.8. Módulo: `reports/reportes.py`

Responsabilidad	Reportes y consultas especiales avanzadas
Funciones	<code>equipos_sin_mantenimiento()</code>: LEFT JOIN para encontrar equipos sin OTs completadas. <code>top_equipos_intervenidos(n=2)</code>: GROUP BY + COUNT + ORDER BY DESC LIMIT 2. <code>tecnico_mayor_costo()</code>: SUM(costo_repuestos) agrupado por técnico. <code>ordenes_atrasadas()</code>: Filtra OTs pendientes/en proceso con más de 7 días usando julianday().

5.9. Módulo: `app.py` (Controlador Web)

Responsabilidad Framework	Servidor web Flask — Punto de entrada de la interfaz gráfica Flask 3.x
Rutas	/ — Dashboard con estadísticas y resumen. /equipos — CRUD completo de equipos (listar, crear, editar, eliminar). /tecnicos — CRUD completo de técnicos. /ordenes — CRUD avanzado de OTs (crear, editar, completar, cancelar, eliminar). /historial — Historial con filtros dinámicos por equipo, técnico y fechas. /reportes — Los 4 reportes especiales.

5.10. Módulo: `views/menu.py` (Controlador Consola)

Responsabilidad	Menús interactivos de la interfaz de consola
Funciones	menu_principal(): Navegación principal con 5 secciones. menu_equipos(): Submenú CRUD de equipos con formularios de entrada. menu_tecnicos(): Submenú CRUD de técnicos. menu_ordenes(): Submenú CRUD avanzado de OTs (8 opciones). menu_historial(): Submenú de historial con 4 tipos de filtro. menu_reportes(): Submenú de reportes especiales.

5.11. Módulo: `views/tabla.py`

Responsabilidad	Utilidades de formateo y presentación para la consola
Funciones	imprimir_tabla(headers, rows): Formatea datos en tabla ASCII alineada. imprimir_detalle(campos): Muestra un registro en formato clave-valor. imprimir_titulo(titulo): Encabezado decorado para secciones. limpiar_pantalla(): Limpia la consola (compatible Windows/-Linux). pausar(): Espera confirmación del usuario (Enter). confirmar(mensaje): Solicita confirmación sí/no.

6 Datos de Ejemplo

El archivo schema.sql incluye datos precargados para pruebas inmediatas:

Tabla	Registros	Descripción
equipos	15	Equipos industriales variados (compresores, tornos CNC, bombas, generadores, soldadoras, etc.) con tres estados diferentes.
tecnicos	12	Técnicos con especialidades diversas (mecánica, electricidad, soldadura, HVAC, automatización, etc.)
ordenes_trabajo	15	9 completadas, 2 en proceso, 3 pendientes (2 atrasadas) y 1 cancelada. Costos variados para reportes significativos.

Cuadro 7: Resumen de datos de ejemplo precargados

Los datos están diseñados para que todos los reportes especiales arrojen resultados significativos:

- Los equipos EQ-013, EQ-014 y EQ-015 no tienen OTs (reporte de equipos sin mantenimiento).
- Los equipos EQ-001 y EQ-002 tienen múltiples intervenciones (reporte top 2).
- Hay OTs con más de 7 días sin completar (reporte de atrasadas).